

四旋翼最短时间航点轨迹优化与流形 MPC

当前工程实现、数学推导、数据流和代码逐项对应

对应 `gpttraj.cpp/.h`、`gptmpc.cpp/.h` 与可视化节点

2026 年 8 月 10 日

目录

1 文档范围与当前实现结论	4
2 坐标系、状态和控制输入	4
2.1 坐标约定	4
2.2 优化状态和底层输入	5
3 数据输入、处理和输出总流程	5
3.1 配置输入	5
3.2 完整数据流	5
4 四旋翼动力学模型	6
4.1 连续时间模型	6
4.2 离散动力学	6
4.3 $SO(3)$ 上的局部误差	7
5 轨迹优化的原始最短时间问题	7
6 固定时间 SCP 子问题	7
6.1 动力学线性化	7
6.2 决策向量	8
6.3 内层目标函数的正确含义	8
7 边界、输入与信赖域约束	9
7.1 初末状态	9
7.2 真实控制输入边界	9
7.3 可选输入变化率约束	9

目录	2
7.4 SCP 信赖域	10
8 CSTC 航点和顺序约束	10
8.1 进度变量	10
8.2 航点空间互补	10
8.3 航点顺序	11
8.4 航点时刻恢复	11
8.5 活跃集锁定	11
9 初值、热启动和最短时间二分	11
9.1 几何初值	11
9.2 初始恢复锚点	12
9.3 时间缩放热启动	12
9.4 外层二分	12
10 可行性判据与 incumbent	13
11 OSQP 与 Eigen 的矩阵构造	13
12 轨迹结果、连续采样与 RViz	14
12.1 轨迹输出	14
12.2 连续采样	14
12.3 可视化数据处理	15
13 在线流形 MPC	15
13.1 参考轨迹输入	15
13.2 MPC 误差状态	15
13.3 预测模型凝聚	15
13.4 MPC 目标和约束	16
14 算法伪代码	16
14.1 轨迹优化	16
14.2 固定时间 SCP	17
14.3 在线 MPC	18
15 公式与代码一对一对应表	18
16 默认配置和输出解释	19

目 录	3
17 功能边界与使用注意事项	20
18 总结	21

1 文档范围与当前实现结论

本文严格描述当前工程中的四个核心文件：

- `include/px4ctrl/gpttraj.h`: 离线轨迹优化输入、输出和配置结构；
- `src/gpttraj.cpp`: 流形动力学、CSTC、SCP、OSQP 和最短时间二分；
- `include/px4ctrl/gptmpc.h`: 在线 MPC 接口、权重和诊断结构；
- `src/gptmpc.cpp`: 轨迹采样、预测模型、凝聚 QP 与控制输出。

同时说明 `gpttraj_visualizer_node.cpp` 和 `config/gpttraj.yaml` 的数据输入、日志和 RViz 输出。

当前默认算法不是把时间、平滑性和能量混合成一个加权性能指标，而是采用字典序结构：

$$T^* = \inf\{T \mid \mathcal{F}(T) \neq \emptyset\} \quad (1)$$

其中 $\mathcal{F}(T)$ 是固定总时间 T 时，由动力学、边界条件、控制输入边界和有序航点约束定义的可行集。外层只最小化 T ；固定 T 的内层代价只用于找到可行点和稳定数值迭代，因此不能用更长时间换取更平滑的控制。

默认实现分成两个航点阶段：

1. 用完整 λ_k^j, μ_k^j CSTC 识别每个航点的通过活跃节点；
2. 锁定该互补活跃集，在外层时间二分中把相应航点球作为硬约束。

第二步解决了高维 CSTC 在每次时间试验中反复切换活跃集所造成的假不可行。因此当前结果是已识别 CSTC 活跃集下的最短时间解，而不是对所有可能互补活跃集的全局最优证明。

2 坐标系、状态和控制输入

2.1 坐标约定

轨迹优化器内部采用工程的 ENU 世界坐标系：

- $\mathbf{p}, \mathbf{v} \in \mathbb{R}^3$ 在 ENU 世界系表达；
- $\mathbf{R} \in \text{SO}(3)$ 把机体 FLU 向量变换到 ENU；
- $\mathbf{e}_3 = [0, 0, 1]^\top$ ；
- 机体 +Z 轴在世界系中的方向为 $\mathbf{R}\mathbf{e}_3$ 。

2.2 优化状态和底层输入

连续状态为

$$\mathbf{x} = (\mathbf{p}, \mathbf{v}, \mathbf{R}) \in \mathbb{R}^3 \times \mathbb{R}^3 \times \text{SO}(3). \quad (2)$$

控制输入为

$$\mathbf{u} = \begin{bmatrix} a_T & \omega_x & \omega_y & \omega_z \end{bmatrix}^T \quad (3)$$

其中 a_T 是质量归一化总拉力，单位为 m/s^2 ，方向沿机体 $+Z$ ； ω 是机体系角速度，单位为 rad/s 。

该输入与工程底层接口直接对应。轨迹优化输出 a_T ，在线控制器最终转换成牛顿单位总拉力：

$$f_T = m a_T. \quad (4)$$

角速度则直接写入 `control_setpoint.bodyrates`。

3 数据输入、处理和输出总流程

3.1 配置输入

ROS 2 配置文件采用

```
waypoint_count: M
waypoints:
  point_0: [x0, y0, z0]
  ...
  point_(M-1): [xM, yM, zM]
```

每个航点是一组完整的 $[x, y, z]$ 。因为 ROS 2 参数不支持一般的二维数值数组，代码用 `waypoints.point_j` 的命名向量表达多点输入。

输入还包括：

- 初始和终端位置、速度、偏航；
- 推力上下限和三个角速度上限；
- 时间上下限、二分容差、离散段数 N ；
- SCP 信赖域、可行性容差和 CSTC 参数；
- RViz 显示频率、最终显示采样数和箭头比例。

3.2 完整数据流

1. 可视化节点读取 YAML，构造 `GptTrajectoryBoundary`、`GptTrajectoryWaypoint` 和 `GptTrajectoryOptions`。
2. `GptTrajectoryOptimizer::optimize()` 检查有限数值、航点数量和容差。

3. 用初点、航点和终点的折线生成几何初值。
4. 在候选时间下运行固定时间 SCP/OSQP, 首先识别 CSTC 活跃节点。
5. 保存非线性可行 incumbent 以及完整 λ, μ 。
6. 外层对总时间进行二分; 时间压缩试验使用上一条可行轨迹热启动。
7. 输出 `GptTrajectoryResult`: 总时间、状态序列、航点时刻、残差和日志历史。
8. RViz 将最终结果重采样成高密度红色曲线和黑色推力箭头。
9. `GptMpcControl` 按控制周期从轨迹连续采样参考, 求解在线 QP。
10. 在线输出牛顿总拉力、机体角速度和期望姿态。

4 四旋翼动力学模型

4.1 连续时间模型

代码使用以总拉力加速度和角速度为输入的降阶四旋翼模型:

$$\dot{\mathbf{p}} = \mathbf{v}, \quad (5a)$$

$$\dot{\mathbf{v}} = -g\mathbf{e}_3 + a_T \mathbf{R} \mathbf{e}_3, \quad (5b)$$

$$\dot{\mathbf{R}} = \mathbf{R}[\boldsymbol{\omega}]_{\times}. \quad (5c)$$

反对称矩阵定义为

$$[\mathbf{a}]_{\times} \mathbf{b} = \mathbf{a} \times \mathbf{b}. \quad (6)$$

对应代码是两个文件中的 `hat()`、`expSo3()` 和 `logSo3()`。

模型没有显式包含电机转速、转动惯量和力矩动力学, 因为这里假定下一级角速度环能够跟踪 $\boldsymbol{\omega}$ 指令。若需要描述电机带宽, 应开启输入变化率约束, 或者把电机/角速度环状态加入模型。

4.2 离散动力学

设总时间为 T , 离散段数为 N :

$$\Delta t = \frac{T}{N}. \quad (7)$$

零阶保持与前向欧拉离散为

$$\mathbf{p}_{k+1} = \mathbf{p}_k + \Delta t \mathbf{v}_k, \quad (8a)$$

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \Delta t (-g\mathbf{e}_3 + a_{T,k} \mathbf{R}_k \mathbf{e}_3), \quad (8b)$$

$$\mathbf{R}_{k+1} = \mathbf{R}_k \text{Exp}(\Delta t \boldsymbol{\omega}_k). \quad (8c)$$

该公式一对一对应 `gpttraj.cpp::propagate()` 和 `gptmpc.cpp::linearizeStep()` 内部的传播表达式。

4.3 SO(3) 上的局部误差

名义状态为 $\bar{\mathbf{x}} = (\bar{\mathbf{p}}, \bar{\mathbf{v}}, \bar{\mathbf{R}})$ 。代码不用九个矩阵元素直接优化姿态，而使用三维李代数误差：

$$\delta \mathbf{x} = \begin{bmatrix} \mathbf{p} - \bar{\mathbf{p}} \\ \mathbf{v} - \bar{\mathbf{v}} \\ \text{Log}(\bar{\mathbf{R}}^\top \mathbf{R}) \end{bmatrix} \in \mathbb{R}^9. \quad (9)$$

求得姿态增量 $\delta \boldsymbol{\theta}$ 后，通过

$$\mathbf{R}^+ = \bar{\mathbf{R}} \text{Exp}(\delta \boldsymbol{\theta}) \quad (10)$$

回缩到 SO(3)。对应 `difference()` 和接受 SCP 解时的 `states[k].r = states[k].r * expSo3(...)`。

5 轨迹优化的原始最短时间问题

理想非线性问题为

$$\begin{aligned} \min_{T, \{\mathbf{x}_k, \mathbf{u}_k\}} \quad & T \\ \text{s.t.} \quad & \mathbf{x}_{k+1} = f_d(\mathbf{x}_k, \mathbf{u}_k, T/N), \\ & \mathbf{x}_0 = \mathbf{x}_{\text{init}}, \quad \mathbf{x}_N = \mathbf{x}_{\text{terminal}}, \\ & a_{T,\min} \leq a_{T,k} \leq a_{T,\max}, \\ & |\omega_{i,k}| \leq \omega_{i,\max}, \\ & \text{依次进入所有航点球,} \\ & T_{\min} \leq T \leq T_{\max}. \end{aligned} \quad (11)$$

默认没有速度上限、倾角上限、能量代价或轨迹平滑性能指标。因此，当最短时间边界由推力决定时，最优解应主动接近 $a_{T,\max}$ ；当前验证结果中推力利用率达到 100%。

6 固定时间 SCP 子问题

6.1 动力学线性化

固定 T 后，在名义轨迹 $(\bar{\mathbf{x}}_k, \bar{\mathbf{u}}_k)$ 附近对离散动力学残差数值差分：

$$\delta \mathbf{x}_{k+1} = \mathbf{A}_k \delta \mathbf{x}_k + \mathbf{B}_k \delta \mathbf{u}_k - \mathbf{d}_k + \boldsymbol{\nu}_k. \quad (12)$$

其中

$$\mathbf{A}_k = \frac{\partial f_d}{\partial \mathbf{x}}, \quad \mathbf{B}_k = \frac{\partial f_d}{\partial \mathbf{u}}, \quad \mathbf{d}_k = \text{当前非线性离散缺陷}. \quad (13)$$

$\boldsymbol{\nu}_k \in \mathbb{R}^9$ 是虚拟控制，用于避免早期线性化 QP 无解；它受到高权重惩罚，且最终必须低于动力学容差。对应 `linearize()` 和动力学等式行的 Triplet 组装。

6.2 决策向量

在完整 CSTC 识别阶段，单轮 QP 的决策变量为

$$\mathbf{z} = \begin{bmatrix} \delta \mathbf{x}_{0:N} \\ \delta \mathbf{u}_{0:N-1} \\ \boldsymbol{\nu}_{0:N-1} \\ \delta \boldsymbol{\lambda} \\ \delta \boldsymbol{\mu} \\ \mathbf{s}^{\text{wp}} \\ \mathbf{s}^{\text{ord}} \end{bmatrix}. \quad (14)$$

各部分维数为

变量	维数	作用
$\delta \mathbf{x}$	$9(N+1)$	位置、速度、姿态局部增量
$\delta \mathbf{u}$	$4N$	推力加速度和角速度增量
$\boldsymbol{\nu}$	$9N$	动力学虚拟控制
$\delta \boldsymbol{\lambda}$	$(N+1)M$	航点剩余进度增量
$\delta \boldsymbol{\mu}$	NM	航点进度消耗增量
$\mathbf{s}^{\text{wp}}$	NM	空间 CSTC 非负松弛
$\mathbf{s}^{\text{ord}}$	$N(M-1)$	严格次序非负松弛

活跃集锁定后的时间二分阶段不再把 λ, μ 和 CSTC 松弛放入 QP，决策向量缩减为

$$\mathbf{z}_{\text{active}} = [\delta \mathbf{x}_{0:N}, \delta \mathbf{u}_{0:N-1}, \boldsymbol{\nu}_{0:N-1}]^{\text{T}}. \quad (15)$$

6.3 内层目标函数的正确含义

固定 T 的 OSQP 目标写成

$$\begin{aligned} J_{\text{inner}} = & \rho_x \sum_k \|\delta \mathbf{x}_k\|_2^2 + \rho_u \sum_k \|\bar{\mathbf{u}}_k + \delta \mathbf{u}_k - \mathbf{u}_h\|_2^2 \\ & + \rho_{\Delta u} \sum_{k=1}^{N-1} \|(\bar{\mathbf{u}}_k + \delta \mathbf{u}_k) - (\bar{\mathbf{u}}_{k-1} + \delta \mathbf{u}_{k-1})\|_2^2 \\ & + \rho_{\nu} \sum_k \|\boldsymbol{\nu}_k\|_2^2 + \rho_{\mu} \sum_{k,j} (\delta \mu_k^j)^2 \\ & + \rho_s \sum_{k,j} (s_{k,j}^{\text{wp}} + s_{k,j}^{\text{ord}}), \end{aligned} \quad (16)$$

其中

$$\mathbf{u}_h = [g, 0, 0, 0]^{\text{T}}. \quad (17)$$

式 (16) 不含 T ，因为 T 在单次内层问题中是常数。它只在同一个 T 的可行集合内选择数值较稳定的解。外层若判定更短的 T 可行，就必然选择更短时间，不会因为式 (16) 更小而保留较长时间。这正是字典序结构与原先联合加权目标的本质区别。

展开式 (16) 后得到 OSQP 标准二次目标：

$$\min_z \quad \frac{1}{2} \mathbf{z}^\top \mathbf{P} \mathbf{z} + \mathbf{q}^\top \mathbf{z}. \quad (18)$$

对应代码中的 `h_triplets` 和 `gradient`。

7 边界、输入与信赖域约束

7.1 初末状态

初始状态增量固定为零：

$$\delta \mathbf{x}_0 = \mathbf{0}. \quad (19)$$

终端增量消除名义终端状态与目标终端状态之间的误差：

$$\delta \mathbf{x}_N = \begin{bmatrix} \mathbf{p}_f - \bar{\mathbf{p}}_N \\ \mathbf{v}_f - \bar{\mathbf{v}}_N \\ \text{Log}(\bar{\mathbf{R}}_N^\top \mathbf{R}_f) \end{bmatrix}. \quad (20)$$

7.2 真实控制输入边界

QP 变量是控制增量，但约束施加在更新后的真实控制上：

$$a_{T,\min} - \bar{a}_{T,k} \leq \delta a_{T,k} \leq a_{T,\max} - \bar{a}_{T,k}, \quad (21a)$$

$$-\boldsymbol{\omega}_{\max} - \bar{\boldsymbol{\omega}}_k \leq \delta \boldsymbol{\omega}_k \leq \boldsymbol{\omega}_{\max} - \bar{\boldsymbol{\omega}}_k. \quad (21b)$$

默认值为

$$a_{T,\max} = 39.22 \text{ m/s}^2, \quad \boldsymbol{\omega}_{\max} = [14, 14, 14]^\top \text{ rad/s}.$$

7.3 可选输入变化率约束

若配置开启，可加入

$$|u_{k,i} - u_{k-1,i}| \leq \dot{u}_{i,\max} \Delta t. \quad (22)$$

还可约束首末输入从悬停输入按同一变化率进入和退出。当前默认关闭这两类约束，所以最终轨迹可能具有很大的 $\dot{a}_T$ 和 $\dot{\boldsymbol{\omega}}$ ；这符合论文层级 $[a_T, \boldsymbol{\omega}]$ 直接作为输入的模型，但真实飞行前必须确认底层执行器带宽。

7.4 SCP 信赖域

每轮局部增量满足

$$\|\delta \mathbf{p}_k\|_\infty \leq r_p, \quad (23a)$$

$$\|\delta \mathbf{v}_k\|_\infty \leq r_v, \quad (23b)$$

$$\|\delta \boldsymbol{\theta}_k\|_\infty \leq r_R. \quad (23c)$$

默认 $r_p = 1$ 、 $r_v = 3$ 、 $r_R = 0.7$ 。这些是 SCP 局部模型有效性的数值约束，不是最终轨迹的物理位置、速度或倾角上限。

8 CSTC 航点和顺序约束

8.1 进度变量

对第 j 个航点，引入

- $\lambda_k^j \in [0, 1]$: 节点 k 时尚未完成的剩余进度;
- $\mu_k^j \in [0, 1]$: 区间 k 消耗的进度。

递推为

$$\lambda_{k+1}^j = \lambda_k^j - \mu_k^j, \quad (24)$$

边界为

$$\lambda_0^j = 1, \quad \lambda_N^j = 0. \quad (25)$$

因此

$$\sum_{k=0}^{N-1} \mu_k^j = 1. \quad (26)$$

8.2 航点空间互补

第 j 个航点球定义为

$$h_k^j = \left\| \mathbf{p}_k - \mathbf{p}_j^{\text{wp}} \right\|_2^2 - r_j^2. \quad (27)$$

CSTC 条件为

$$\mu_k^j h_k^j \leq 0. \quad (28)$$

因为 $\mu_k^j \geq 0$ ，当 $\mu_k^j > 0$ 时，必须有 $h_k^j \leq 0$ ，即飞机进入航点球。

在名义点 $(\bar{\mu}_k^j, \bar{\mathbf{p}}_k)$ 对乘积一阶展开：

$$\bar{\mu}_k^j \bar{h}_k^j + \bar{h}_k^j \delta \mu_k^j + 2\bar{\mu}_k^j (\bar{\mathbf{p}}_k - \mathbf{p}_j^{\text{wp}})^\top \delta \mathbf{p}_k - s_{k,j}^{\text{wp}} \leq 0, \quad (29)$$

其中 $s_{k,j}^{\text{wp}} \geq 0$ 。该式直接对应 `waypoint_cstc_rows` 的稀疏矩阵行。

8.3 航点顺序

弱顺序约束为

$$\lambda_k^j \leq \lambda_k^{j+1}. \quad (30)$$

它表示前一个航点的剩余进度不能大于后一个航点，从而避免完成顺序逆转。

严格顺序互补为

$$\lambda_k^j \mu_k^{j+1} = 0. \quad (31)$$

当前一航点仍未完成，即 $\lambda_k^j > 0$ 时，后一个航点不能开始消耗进度。一阶展开为

$$\bar{\mu}_k^{j+1} \delta \lambda_k^j + \bar{\lambda}_k^j \delta \mu_k^{j+1} + \bar{\lambda}_k^j \bar{\mu}_k^{j+1} - s_{k,j}^{\text{ord}} \leq 0. \quad (32)$$

8.4 航点时刻恢复

完整 CSTC 阶段通过 μ 的重心恢复航点时刻：

$$t_j = \frac{T}{N} \sum_{k=0}^{N-1} k \mu_k^j. \quad (33)$$

代码同时保存完整 `progress_lambda` 和 `progress_mu`，避免只保存重心后丢失进度分布。

8.5 活跃集锁定

完整 CSTC 得到可行解后，以式 (33) 对应的离散节点作为互补活跃节点。在后续时间二分中固定该节点，并施加分量约束

$$|p_{k_j,i} - p_{j,i}^{\text{wp}}| \leq \frac{r_j}{\sqrt{3}}, \quad i \in \{x, y, z\}. \quad (34)$$

由

$$\sum_i (p_{k_j,i} - p_{j,i}^{\text{wp}})^2 \leq 3 \left(\frac{r_j}{\sqrt{3}} \right)^2 = r_j^2$$

可知式 (34) 是原航点球的内接立方体约束，因此严格保证进入航点球。它比球约束略保守，但保持 QP 线性，并消除了互补模式切换。

9 初值、热启动和最短时间二分

9.1 几何初值

设初点、全部航点和终点的折线长度为 L 。初始候选时间为

$$T_0 = \text{clip} \left(\frac{1.5L}{v_{\text{init}}}, T_{\min}, T_{\max} \right). \quad (35)$$

位置按折线累计弧长均匀布点，速度由中心差分初始化。姿态的机体 Z 轴与所需合力方向一致：

$$\mathbf{z}_{b,k} = \frac{\mathbf{a}_k + g\mathbf{e}_3}{\|\mathbf{a}_k + g\mathbf{e}_3\|_2}. \quad (36)$$

初始推力与角速度为

$$\mathbf{a}_{T,k} = (\mathbf{a}_k + g\mathbf{e}_3)^\top \mathbf{R}_k \mathbf{e}_3, \quad (37a)$$

$$\boldsymbol{\omega}_k = \frac{1}{\Delta t} \text{Log}(\mathbf{R}_k^\top \mathbf{R}_{k+1}). \quad (37b)$$

9.2 初始恢复锚点

为了稳定第一条 CSTC 可行种子, 代码可在仅初始恢复阶段临时使用输入变化率和首尾悬停输入锚点。该锚点在所有最短时间试验中关闭, 除非用户在配置中显式开启; 因此它不会缩小最终最短时间问题的可行集。

9.3 时间缩放热启动

若已有总时间 T_{old} 的可行轨迹, 测试较短时间 T_{new} 时保持归一化路径参数 $s = t/T$:

$$\mathbf{p}_{\text{new}}(s) = \mathbf{p}_{\text{old}}(s). \quad (38)$$

令

$$\alpha = \frac{T_{\text{old}}}{T_{\text{new}}}, \quad (39)$$

则速度热启动为

$$\mathbf{v}_{\text{new}}(s) = \alpha \mathbf{v}_{\text{old}}(s). \quad (40)$$

姿态与输入根据缩放后的加速度重新构造, 以正确处理重力项。完整 λ, μ 或已识别的活跃节点沿归一化时间保持。

9.4 外层二分

首先扩大候选时间直到获得可行上界 T_F :

$$T \leftarrow \min(1.35T, T_{\text{max}}). \quad (41)$$

随后测试 T_{min} 。若不可行, 设置

$$T_I = T_{\text{min}}, \quad T_F = T_{\text{current}},$$

并迭代

$$T_{\text{trial}} = \frac{T_I + T_F}{2}. \quad (42)$$

若固定时间问题可行, 则

$$T_F \leftarrow T_{\text{trial}};$$

否则

$$T_I \leftarrow T_{\text{trial}}.$$

当

$$T_F - T_I \leq \varepsilon_T \quad (43)$$

或达到最大搜索次数时, 返回最短已知可行上界 T_F 。

10 可行性判据与 incumbent

每轮接受更新后，代码重新用非线性传播式 (8) 计算真实残差。固定时间轨迹被判为可行需要

$$\max_k \|\boldsymbol{\nu}_k\|_\infty < \varepsilon_{\text{dyn}}, \quad (44a)$$

$$\max_k \|f_d(\mathbf{x}_k, \mathbf{u}_k, \Delta t) - \mathbf{x}_{k+1}\|_\infty < \varepsilon_{\text{dyn}}, \quad (44b)$$

$$r_{\text{wp}} < \varepsilon_{\text{CSTC}}, \quad (44c)$$

$$r_{\text{ord}} < \varepsilon_{\text{CSTC}}, \quad (44d)$$

$$s_{\text{max}} < \varepsilon_{\text{slack}}. \quad (44e)$$

只要某一轮满足式 (44)，就保存为可行 incumbent。相同 T 下，代码用归一化残差 merit 选择更好的可行迭代，而不是像旧实现一样永远返回第一个可行迭代。

`success=true` 表示式 (44) 成立。`converged=true` 还要求

$$\|\Delta \mathbf{z}\|_\infty \leq \varepsilon_{\text{update}}. \quad (45)$$

因此 `success=true`, `converged=false` 表示已经有约束可行轨迹，但尚未证明 SCP 驻点收敛；`Optimization failed` 则表示没有任何迭代满足可行性阈值。

11 OSQP 与 Eigen 的矩阵构造

OSQP 接收

$$\begin{aligned} \min_{\mathbf{z}} \quad & \frac{1}{2} \mathbf{z}^\top \mathbf{P} \mathbf{z} + \mathbf{q}^\top \mathbf{z}, \\ \text{s.t.} \quad & \mathbf{l} \leq \mathbf{A} \mathbf{z} \leq \mathbf{u}. \end{aligned} \quad (46)$$

轨迹优化中的约束行按如下顺序组装：

1. 初始状态和终端状态等式；
2. N 段线性化动力学等式；
3. CSTC 进度递推与进度边界；
4. 弱顺序、空间 CSTC 和严格顺序；
5. 或者锁定活跃集后的固定航点分量约束；
6. 可选输入变化率约束；
7. 单位矩阵行：信赖域、控制边界、进度边界和松弛非负性。

代码用 `Eigen::Triplet<double>` 累积非零元素，然后构造 `Eigen::SparseMatrix<double>`。`toCsc()` 把矩阵压缩成 CSC：

- $\mathbf{P}$ 只传上三角；

- $\mathbf{A}$ 传完整稀疏矩阵;
- $\mathbf{q}, \mathbf{l}, \mathbf{u}$ 从 Eigen 连续内存复制到 OSQP 数组;
- OSQP 解再复制回 `Eigen::VectorXd`。

轨迹优化 OSQP 默认最大迭代数为 50000, 绝对和相对容差均为 5×10^{-5} , 并开启 polishing。

12 轨迹结果、连续采样与 RViz

12.1 轨迹输出

`GptTrajectoryResult` 输出:

- `success`、`converged` 和文字状态;
- 总时间 T 、SCP 次数和总求解时间;
- $N + 1$ 个 `GptTrajectoryState`;
- 每个航点的选定时刻;
- 完整 λ, μ ;
- 动力学、虚拟控制、航点、顺序和松弛残差;
- 所有接受的 SCP 历史帧。

12.2 连续采样

对 $t_k \leq t < t_{k+1}$, 定义

$$\beta = \frac{t - t_k}{t_{k+1} - t_k}. \quad (47)$$

位置、速度、推力和角速度线性插值:

$$\mathbf{y}(t) = (1 - \beta)\mathbf{y}_k + \beta\mathbf{y}_{k+1}. \quad (48)$$

姿态使用四元数最短路径 SLERP:

$$\mathbf{q}(t) = \text{SLERP}(\mathbf{q}_k, \mathbf{q}_{k+1}, \beta). \quad (49)$$

对应 `GptTrajectoryOptimizer::sample()`。MPC 因此按控制周期采样, 不受优化网格 $N = 60$ 的 12.4 Hz 节点频率限制。

12.3 可视化数据处理

橙色曲线直接显示每一轮 SCP 的优化节点。最终红色轨迹独立重采样为默认 240 段、241 个显示状态；该操作不增加 OSQP 决策变量。

每个黑色箭头起点为 $\mathbf{p}_k$ ，终点为

$$\mathbf{p}_k^{\text{arrow}} = \mathbf{p}_k + c_f a_{T,k} \mathbf{R}_k \mathbf{e}_3, \quad c_f = 0.03. \quad (50)$$

因此箭头方向是机体 Z 轴/推力方向，长度与 a_T 成正比。比例从 0.06 改为 0.03 后，所有箭头等比例缩短一半，相对大小保持不变。

蓝色长方体使用 `sample()` 沿最终轨迹循环，姿态来自轨迹四元数，移动速度由轨迹时间和 `vehicle_playback_speed` 决定。

13 在线流形 MPC

13.1 参考轨迹输入

若调用 `setTrajectory()`，在线时刻 t 和预测步 k 的参考为

$$\mathbf{r}_k = \text{sample}(\text{trajectory}, t + k\Delta t_{\text{MPC}}). \quad (51)$$

若没有完整轨迹但提供 `preview`，则直接使用 `preview`；否则根据 `Ref_State_t` 的位置、速度、加速度和 `jerk` 构造动态一致参考。

13.2 MPC 误差状态

当前状态相对第一个参考的误差为

$$\mathbf{e}_0 = \begin{bmatrix} \mathbf{p} - \mathbf{p}_0^d \\ \mathbf{v} - \mathbf{v}_0^d \\ \text{Log}((\mathbf{R}_0^d)^\top \mathbf{R}) \end{bmatrix} \in \mathbb{R}^9. \quad (52)$$

控制增量为

$$\delta \mathbf{u}_k = \mathbf{u}_k - \mathbf{u}_k^d \in \mathbb{R}^4. \quad (53)$$

在每个参考点数值线性化：

$$\mathbf{e}_{k+1} = \mathbf{A}_k \mathbf{e}_k + \mathbf{B}_k \delta \mathbf{u}_k + \mathbf{c}_k. \quad (54)$$

13.3 预测模型凝聚

堆叠未来误差

$$\mathbf{E} = [\mathbf{e}_1^\top, \dots, \mathbf{e}_H^\top]^\top$$

和控制增量

$$\Delta \mathbf{U} = [\delta \mathbf{u}_0^\top, \dots, \delta \mathbf{u}_{H-1}^\top]^\top.$$

递推式 (54) 得到

$$\mathbf{E} = \mathbf{S}_x \mathbf{e}_0 + \mathbf{S}_u \Delta \mathbf{U} + \mathbf{c}. \quad (55)$$

对应代码中的 `sx`、`su`、`affine`、`phi` 和 `gamma`。

13.4 MPC 目标和约束

在线性能指标为

$$J_{\text{MPC}} = \mathbf{E}^\top \bar{\mathbf{Q}} \mathbf{E} + \Delta \mathbf{U}^\top \bar{\mathbf{R}} \Delta \mathbf{U}. \quad (56)$$

代入式 (55):

$$\begin{aligned} \mathbf{P}_{\text{MPC}} &= 2(\mathbf{S}_u^\top \bar{\mathbf{Q}} \mathbf{S}_u + \bar{\mathbf{R}}), \\ \mathbf{q}_{\text{MPC}} &= 2\mathbf{S}_u^\top \bar{\mathbf{Q}} (\mathbf{S}_x \mathbf{e}_0 + \mathbf{c}). \end{aligned} \quad (57)$$

输入增量边界保证真实输入满足

$$a_{T,\min} - a_{T,k}^d \leq \delta a_{T,k} \leq a_{T,\max} - a_{T,k}^d, \quad (58a)$$

$$-\omega_{\max} - \omega_k^d \leq \delta \omega_k \leq \omega_{\max} - \omega_k^d. \quad (58b)$$

约束矩阵是单位矩阵。OSQP 成功后只执行第一步控制:

$$\mathbf{u}_{\text{cmd}} = \mathbf{u}_0^d + \delta \mathbf{u}_0^*. \quad (59)$$

若 OSQP 失败, 则使用上一控制并裁剪推力边界。

14 算法伪代码

14.1 轨迹优化

```

Input:
  initial boundary, terminal boundary
  ordered waypoint balls
  thrust/body-rate bounds and numerical options

Validate input
Build polyline geometric initialization
T_candidate <- clip(1.5 * path_length / initial_speed)

repeat
  SolveFixedTime(T_candidate, CSTC enabled, restoration anchor enabled)
  if feasible: break
  T_candidate <- min(1.35 * T_candidate, T_max)
until T_candidate = T_max

if no feasible CSTC seed:
  return failure

```

```

Save CSTC passage active nodes and feasible incumbent
T_feasible <- T_candidate

Test T_min using time-dilated warm start and locked active nodes
if T_min feasible:
    return T_min trajectory

T_infeasible <- T_min
while T_feasible - T_infeasible > time_search_tolerance:
    T_trial <- 0.5 * (T_feasible + T_infeasible)
    trial <- SolveFixedTime(
        T_trial,
        warm_start = shortest feasible trajectory,
        CSTC active nodes locked)
    if trial feasible:
        save trial as new incumbent
        T_feasible <- T_trial
    else:
        T_infeasible <- T_trial

return shortest feasible incumbent and final time bracket

```

14.2 固定时间 SCP

```

Initialize states, controls, and optionally lambda/mu
for SCP iteration = 1 ... max_scp_iterations:
    Numerically linearize nonlinear dynamics
    Build sparse Hessian P and gradient q
    Build boundary, dynamics, waypoint, input, trust-region rows
    Solve  $l \leq A z \leq u$  with OSQP
    if OSQP failed:
        return current fixed-time attempt as failed

    Retract state update onto  $R^3 \times R^3 \times S^2(3)$ 
    Update controls and CSTC progress
    Recompute nonlinear dynamics and CSTC residuals
    if all nonlinear feasibility tests pass:
        save or improve feasible incumbent
    publish iteration timing, residuals, states, and controls
    if update is small and residuals are feasible:
        mark converged and stop

return best feasible incumbent, otherwise failure

```

14.3 在线 MPC

```

At every control callback:
  Sample H+1 references from optimized trajectory
  Compute current manifold error e0
  Linearize H reference transitions
  Condense prediction:  $E = S_x e_0 + S_u \Delta U + c$ 
  Build  $P = 2(S_u' Q S_u + R)$ 
  Build  $q = 2 S_u' Q(S_x e_0 + c)$ 
  Build thrust/body-rate increment bounds
  Solve the OSQP
  Apply only  $u_{\text{ref}}[0] + \Delta U[0]$ 
  Convert aT to Newton thrust:  $fT = \text{mass} * aT$ 
  Publish thrust, body rates, desired attitude, and diagnostics

```

15 公式与代码一对一对应表

编号	数学内容	当前代码位置
F1	反对称矩阵式 (6)	<code>gpttraj.cpp::hat()</code> ; <code>gptmpc.cpp::hat()</code>
F2	SO(3) 指数、对数与回缩式 (10)	两文件的 <code>expSo3()</code> 、 <code>logSo3()</code> ; SCP 状态更新
F3	连续模型式 (5)	<code>gpttraj.h</code> 顶部输入说明; 两个传播/线性化函数
F4	离散模型式 (8)	<code>gpttraj.cpp::propagate()</code>
F5	局部误差式 (9)	<code>gpttraj.cpp::difference()</code>
F6	线性动力学式 (12)	<code>gpttraj.cpp::linearize()</code> 和 <code>dynamics rows</code>
F7	固定时间目标式 (16)	<code>h_triplets</code> 、 <code>gradient</code> 构造块
F8	初末边界式 (19)–(20)	QP 的前 18 个 equality rows
F9	控制边界式 (21)	<code>identity-bound rows</code> 中 <code>input variables</code> 分支
F10	可选变化率式 (22)	<code>add_rate_rows()</code>
F11	信赖域式 (23)	<code>identity-bound rows</code> 中 <code>state variables</code> 分支
F12	进度递推式 (24)、边界式 (25)	<code>progress rows</code> 和 <code>progress boundary rows</code>
F13	空间 CSTC 式 (29)	<code>waypoint CSTC rows</code>
F14	弱顺序式 (30)	<code>weak order rows</code>
F15	严格顺序式 (32)	<code>strict order CSTC rows</code>
F16	航点时刻式 (33)	填充 <code>result.waypoint_times</code> 的循环
F17	活跃集硬航点式 (34)	<code>fixed_waypoint_rows</code>

编号	数学内容	当前代码位置
F18	热启动式 (38)–(40)	<code>use_warm_start</code> 分支
F19	最短时间式 (1) 与二分式 (42)	<code>optimize()</code> 末尾 <code>minimum-time bisection</code>
F20	可行性式 (44)	<code>residuals_feasible</code> 和 <code>feasible incumbent</code>
F21	OSQP 标准式 (46)	<code>solve0sqp()</code> 、 <code>toCsc()</code>
F22	插值式 (48) 和 SLERP (49)	<code>GptTrajectoryOptimizer::sample()</code>
F23	RViz 箭头式 (50)	<code>appendTrajectoryAndThrust()</code>
M1	MPC 参考式 (51)	<code>GptMpcControl::calculateControl()</code> 的 <code>trajectory</code> 分支
M2	MPC 误差式 (52)	<code>gptmpc.cpp::stateError()</code>
M3	MPC 线性化式 (54)	<code>gptmpc.cpp::linearizeStep()</code>
M4	凝聚预测式 (55)	<code>sx</code> 、 <code>su</code> 、 <code>affine</code> 递推
M5	MPC 目标式 (57)	<code>h_dense</code> 和 <code>gradient</code>
M6	MPC 输入边界式 (58)	<code>lower</code> 、 <code>upper</code> 构造循环
M7	滚动控制式 (59)	<code>delta_input.head<4>()</code> 与控制输出
M8	牛顿推力转换式 (4)	<code>control_setpoint.thrust = mass * command(0)</code>

16 默认配置和输出解释

当前关键默认参数为：

参数	默认值
优化离散段数 N	60
最大 SCP 次数	20
最大时间搜索次数	12
时间二分容差	0.02 s
推力加速度范围	$[0.5, 39.22] \text{ m/s}^2$
角速度范围	每轴 $[-14, 14], \text{ rad/s}$
输入变化率约束	关闭
完整 CSTC	开启
CSTC 活跃集锁定	开启
最终显示段数	240
推力箭头比例 c_f	0.03

典型成功日志

```
Optimization succeeded (converged=false):
CSTC-active-set pure-minimum-time bisection,
bracket=[4.813..., 4.826...] s
Control-bound utilization: thrust=100.0%, body-rate=85.5%
```

解释如下：

- **succeeded**: 存在满足当前残差阈值的非线性可行 incumbent；
- **converged=false**: SCP 更新范数尚未达到驻点阈值，不能宣称 KKT 收敛；
- **bracket**: 左端为已知不可行时间，右端为返回的最短已知可行时间；
- 推力利用率 100%: 时间目标确实把控制推到物理配置边界；
- MPC 会按控制周期重新采样，并非只按 $N + 1$ 个优化节点执行。

17 功能边界与使用注意事项

1. **局部而非全局最优**: SCP 和 CSTC 都是非凸问题的局部方法；活跃集锁定后得到的是所识别通过节点下的最短时间区间，不是所有可能航点时刻的全局证明。
2. **输入带宽**: 默认只约束 a_T 和 ω 本身，不约束它们的变化率。当前激进轨迹可能出现很大的 $\dot{a}_T$ 和 $\dot{\omega}$ 。真实飞行前应根据实际飞控和电机带宽开启式 (22)，并重新求解最短时间。
3. **降阶模型**: 模型假定角速度是可直接跟踪的底层输入，未包含电机和刚体角动量方程。若底层角速度环不足以实现参考，应扩展模型或降低角速度/变化率上限。

4. **离散误差：** $N = 60$ 是当前五航点配置下兼顾 CSTC 稳定性与计算量的默认值。提高 N 不一定单调改善非凸求解，反而可能导致进度互补振荡。最终应检查非线性动力学残差，并用更高精度传播器离线复核。
5. **球与内接盒：** 活跃集阶段使用式 (34)，保证进入原航点球，但比完整球稍保守。若需要消除这部分保守性，可在活跃节点固定后使用二阶锥求解器处理球约束。
6. **可视化不等于控制离散：** RViz 的 241 个最终显示点只影响 Marker 密度；MPC 使用连续采样函数和控制周期，两者与轨迹优化决策变量数量相互独立。

18 总结

当前工程形成了完整闭环：

YAML 航点 $\rightarrow$ CSTC 活跃集识别 $\rightarrow$ 固定时间 SCP/OSQP

$\rightarrow$ 外层最短时间二分 $\rightarrow$ 连续轨迹采样 $\rightarrow$ 流形 MPC $\rightarrow [f_T, \omega]$.

轨迹层唯一第一性能指标是总时间；动力学、航点、顺序、输入边界和残差阈值决定可行域；内层正则只在固定时间下稳定求解。在线 MPC 再根据实际状态误差对轨迹的前馈推力和角速度进行有限时域修正，并保持与工程底层接口一致。